

UT 03. El Lenguaje DDL. Tablas y Constraints.



Autor: Carlos Moreno Martínez.
cmorenomartinez@educa.madrid.org



UT 03. El Lenguaje DDL. Tablas y Constraints.

1.- Introducción.



DDL (Data Definition Language).

Permite a realizar tareas de creación, mantenimiento y eliminación de objetos tales como: tablas, vistas, procedimientos, etc.

Diccionario de Datos.

- .Catálogo con los objetos del Sistema.
- .Gestionado por el SGBD.
- .Los usuarios pueden realizar consultas para obtener información de los objetos.

Esquemas.

- .Estructura que sirve para almacenar objetos.
- .En Oracle cada usuario posee un esquema, con el mismo nombre que el usuario, pensado para almacenar sus objetos.** Los usuarios pueden (con los adecuados privilegios) realizar operaciones en objetos de otros esquemas.

Creación de Objetos.

- .Herramientas gráficas: Oracle SQL Developer, DB Beaver, etc.
- .Lineas de comandos: sqlplus.

UT 03. El Lenguaje DDL. Tablas y Constraints.

2.- Operaciones básicas que forman el Lenguaje DDL.

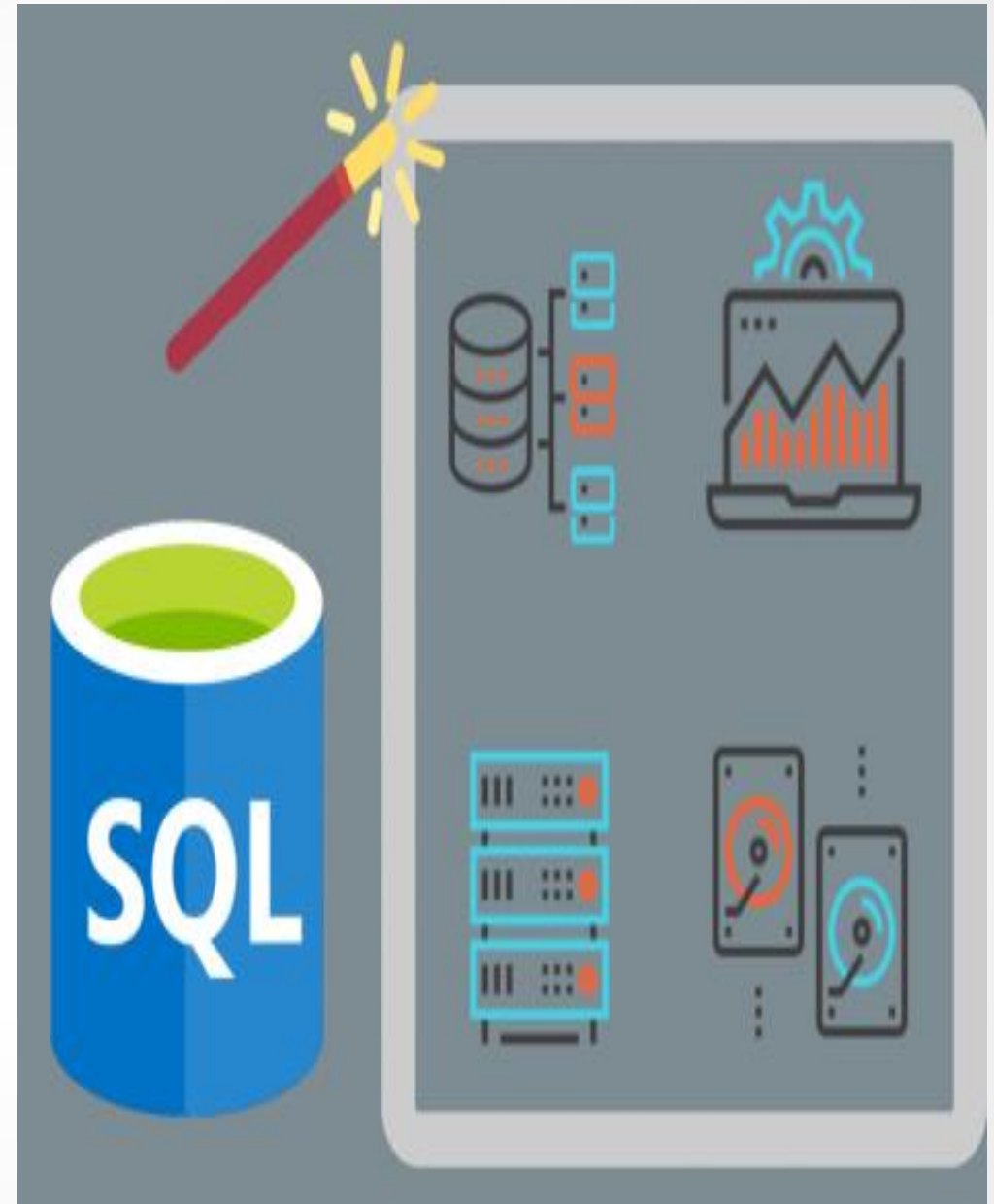
En general podemos decir que el lenguaje DDL está formado por tres tipos de sentencias:

- **CREATE.** Crea un objeto.
- **ALTER.** Modifica un objeto.
- **DROP.** Elimina un objeto.

Dependiendo del SGBD puede haber otras:

• **TRUNCATE.** Vacía de información un objeto,

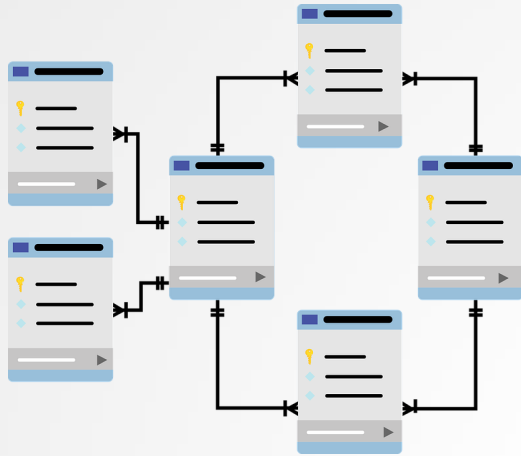
• **RENAME.** Renombra un objeto.



UT 03. El Lenguaje DDL. Tablas y Constraints.

3.- Creación de una tabla.

3.1.- Formato de la Sentencia.



Datos Necesarios para crear una tabla.

- Nombre de la tabla.
- Nombre de cada una de las columnas.
- Tipo y tamaño de datos que se almacenará en cada columna.
- Restricciones sobre los datos.

```
CREATE TABLE [ESQUEMA.]NOMBRETABLA (  
    COLUMNA1 TIPODATO1 [DEFAULT VALOR1] [RESTRICCIONES1],  
    COLUMNA2 TIPODATO2 [DEFAULT VALOR2] [RESTRICCIONES2],  
    ...  
    COLUMNAN TIPODATON [DEFAULT VALORN] [RESTRICCIONESN]  
);
```

Hay que tener en cuenta a la hora de no cometer errores que:

- La definición de los campos de la tabla va entre paréntesis. Es decir se abre paréntesis al principio y se cierra con el último.
- La definición de un campo se cierra con una coma (",").
- La sentencia SQL siempre acaba con un punto y coma (";").

UT 03. El Lenguaje DDL. Tablas y Constraints.

3.- Creación de una tabla.

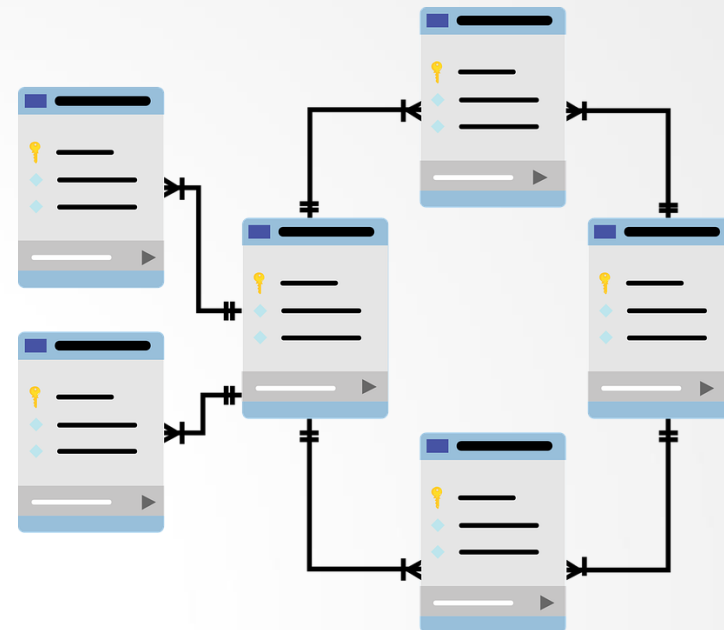
3.2.- Nombres de Tablas y Columnas.

Nombre de la tabla. La tabla se nombrará mediante una cadena de caracteres con los siguientes **requisitos**:

- El nombre debe empezar por un carácter alfabético.
- Su longitud máxima es de **30 caracteres**.
- Sólo se permiten en el nombre letras del **alfabeto inglés, dígitos y el guion bajo (“_”)**.
- No puede ser una palabra reservada de SQL.
- **No se distingue entre mayúsculas y minúsculas.**
- En el mismo esquema no puede haber nombres de tablas duplicados. Sin embargo, puede haber tablas con el mismo nombre localizadas en distintos esquemas.

Nombre de cada una de las columnas.

- **Deben cumplir los mismos requisitos que los nombres de las tablas u otros objetos de la base de datos.**
- En algunos casos irán acompañados del **tamaño del campo** (numéricos, cadenas de caracteres, etc).



UT 03. El Lenguaje DDL. Tablas y Constraints.

3.- Creación de una tabla.

3.3.- Estructura de una tabla y descripción del contenido.

Ejemplo. En este ejemplo vamos a crear la tabla PROVEEDORES con tres campos: PROVEEDORID de tipo numérico con seis dígitos y los campos NOMBRE y CIUDAD que almacenarán cadenas de caracteres con un máximo de treinta caracteres.

```
CREATE TABLE PROVEEDORES (  
    PROVEEDORID    NUMBER(6),  
    NOMBRE          VARCHAR2(30),  
    CIUDAD          VARCHAR2(30) DEFAULT 'MADRID'  
);
```

Table PROVEEDORES creado.

Para visualizar la estructura de una tabla se dispone del comando **DESCRIBE** (se puede reducir a **DESC**). Añadiendo el nombre de la tabla, mostrará una lista de campos con su tipo e información sobre las restricciones que tiene dicho campo.

```
DESCRIBE PROVEEDORES;
```

Nombre	¿Nulo?	Tipo
-----	-----	-----
PROVEEDORID		NUMBER(6)
NOMBRE		VARCHAR2(30)
CIUDAD		VARCHAR2(30)

UT 03. El Lenguaje DDL. Tablas y Constraints.

3.- Creación de una tabla.

3.4.- Tablas presentes en un esquema. La tabla USER_TABLES.

Se pueden consultar el nombre y la información de las tablas que contiene un esquema mediante una consulta a la tabla del diccionario **USER_TABLES**.

```
SELECT TABLE_NAME FROM USER_TABLES;
```

```
TABLE_NAME
```

```
-----
```

```
PROVEEDORES
```

Veamos como la tabla almacena valores. Para ello, vamos a realizar la inserción de dos registros y, posteriormente, realizaremos una consulta sobre la tabla.

```
INSERT INTO PROVEEDORES VALUES (1, 'LOPSA', 'SEVILLA');
```

1 fila insertadas.

```
INSERT INTO PROVEEDORES (PROVEEDORID, NOMBRE) VALUES (2, 'INEPSA');
```

1 fila insertadas.

```
SELECT * FROM PROVEEDORES;
```

PROVEEDORID	NOMBRE	CIUDAD
-----	-----	-----
1	LOPSA	SEVILLA
2	INEPSA	MADRID

UT 03. El Lenguaje DDL. Tablas y Constraints.

4.- Tipos de datos básicos en Oracle.

4.1.- Tipos Alfanuméricos.

Tipo	Descripción
VARCHAR2	Textos de longitud variable, máximo 4000 caracteres . En la base de datos se almacena una cadena con una longitud igual a la longitud del dato.
CHAR	Textos de longitud fija de hasta 2000 caracteres . En la base de datos se almacenará una cadena de la longitud especificada independientemente del texto almacenado.
NCHAR	Caracteres nacionales de longitud fija.
NVARCHAR2	Caracteres nacionales de longitud variable.

.Siempre se indican los tamaños entre paréntesis tras el nombre del tipo.

.Conviene reservar suficiente espacio para reservar los valores, en caso del tipo VARCHAR2, no se pierde espacio por reservar más del deseado ya que si el texto es más pequeño que el tamaño indicado, el espacio sobrante se libera.

UT 03. El Lenguaje DDL. Tablas y Constraints.

4.- Tipos de datos básicos en Oracle.

4.2.- Tipo de datos numéricos.

Tipo	Descripción
NUMBER	Los números decimales en coma flotante en Oracle se almacenan mediante el tipo NUMBER. Permite representar todo tipo de números. Su rango está entre 10^{-130} y $9,99 \times 10^{128}$. Fuera de estos límites Oracle devuelve error.
NUMBER(p,s)	Los números decimales se indican con NUMBER(p,s) , donde: .p. Precisión máxima (número total de dígitos del número). .s. Escala (número de decimales a la derecha de la coma). Por ejemplo, NUMBER(8,3), permite representar hasta números de 8 cifras de las cuales 3 serían decimales (el decimal se representa con un punto). Si la escala es negativa indica ceros a la izquierda del decimal.
NUMBER(p)	Para definir la precisión de un número entero se usa NUMBER(p), donde p indica el número de dígitos que puede tener el número . La precisión máxima de un número es como máximo treinta y ocho.

Número introducido por el usuario: 324587.323

Formato	Valor almacenado	Explicación
NUMBER(9,4)	324587.323	Admite cualquier tipo de valor.
NUMBER(9,2)	324587.32	Redondea a dos decimales.
NUMBER(9)	324587	Redondea al entero más próximo.
NUMBER(9,-2)	324600	Como la escala es menos dos, redondea a la centena.

UT 03. El Lenguaje DDL. Tablas y Constraints.

4.- Tipos de datos básicos en Oracle.

4.3.- Tipos de datos temporales (I). Fecha y hora convencionales.

Tipo	Descripción
DATE	•Permite almacenar fechas que se pueden escribir en formato día, mes y año entre comillas. El separador normalmente es una barra inclinada (“/”) o un guion (“-”). •Para conocer la fecha actual, Oracle proporciona la función “SYSDATE” que devuelven la fecha y hora actual. •En Oracle, en los campos fecha se almacena el valor de un instante indicado mediante fecha y hora. Ejemplo: ‘2/2/2015 18:34:23.18’. •No conviene manejarlos directamente ya que suelen ser representadas en informes en multitud de formatos. Por ello, se usan funciones de conversión de fecha a texto y viceversa para manejarlas.
TIMESTAMP	•Almacena en el campo un valor de un instante indicado mediante fecha y hora. Por tanto, almacena los valores de día, mes y año, junto con hora, minuto y segundos. •Junto con los segundos almacena las fracciones de segundo. Representa un instante concreto en el tiempo. Ejemplo: ‘2/2/2015 18:34:23.18’.

UT 03. El Lenguaje DDL. Tablas y Constraints.

4.- Tipos de datos básicos en Oracle.

4.3.- Tipos de datos temporales (II). Intervalos.

INTERVAL YEAR TO MONTH. Representa intervalos de fecha.		
INTERVAL 'intervalo' YEAR[(precisión)]	INTERVAL '30' YEAR	30 años.
	INTERVAL '123'	Error: la precisión por defecto es dos y el intervalo indicado tiene tres dígitos.
	INTERVAL '123' YEAR(3)	123 años.
INTERVAL 'intervalo' MONTH[(prec.)]	INTERVAL '30' MONTH	30 meses.
INTERVAL 'intervalo' YEAR TO MONTH	INTERVAL '2-6' YEAR TO MONTH	Dos años y seis meses.

INSERT INTO EMPLEADOS

(EMPLEADOID, NOMBRE, EXPERIENCIA) VALUES

(328, 'ANA MARIA GONZALEZ', INTERVAL '3-9' YEAR TO MONTH);

1 fila insertadas.

INTERVAL DAY TO SECOND. Análogo al anterior para mostrar intervalos horarios.	
INTERVAL '130' MINUTE(3)	130 minutos.
INTERVAL '25' HOUR	25 horas.
INTERVAL '10:22' MINUTE TO SECOND	10 minutos y 22 segundos.
INTERVAL '30.12345' SECOND (2,4)	30.1235 segundos (ha sido redondeado a 4 decimales).
INTERVAL '4 5:12' DAY TO MINUTE	4 días, 5 horas y 12 minutos.

UT 03. El Lenguaje DDL. Tablas y Constraints.

4.- Tipos de datos básicos en Oracle.

4.4.- Otros tipos de datos.

Datos de gran tamaño.	
Tipo	Descripción
BLOB.	Datos binarios no estructurados.
CLOB.	Datos de tipo carácter.
NLOB.	Caracteres Unicode.
BFILE.	Almacena datos binarios en archivos de sistema operativo, fuera de la base de datos. La columna definida como BFILE guarda un localizador del archivo externo que contiene los datos.

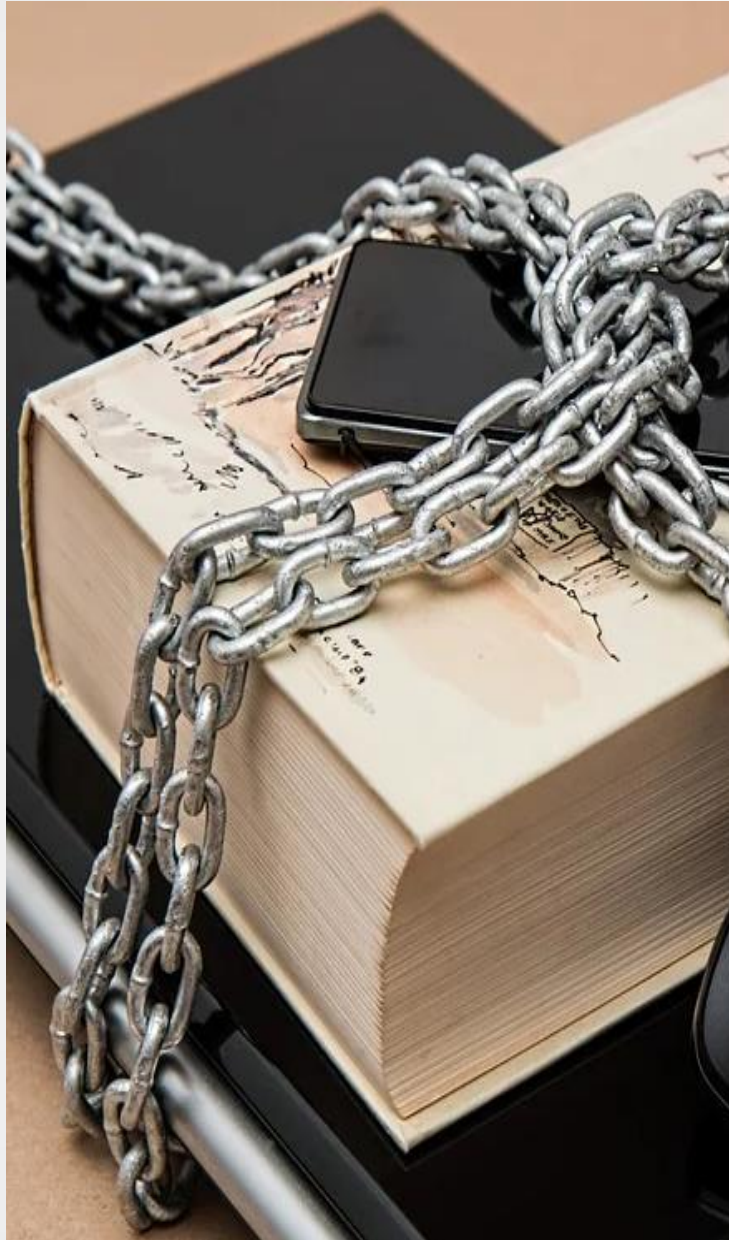
Pseudocolumnas.

•Dependiendo del Sistema Gestor de Bases de Datos, pueden implementarse una serie de pseudocolumnas para obtener datos del Sistema.

•**Ejemplo. ROWID.** Representa una dirección de la base de datos ocupada por una fila determinada. Se puede consultar con la sentencia SELECT, pero no modificar ni borrar.

UT 03. El Lenguaje DDL. Tablas y Constraints.

5.- Restricciones (Constraints).



Concepto.

.Las restricciones (también llamadas constraints) son **limitaciones del dominio de datos que se pueden imponer a los valores de una columna.**

.El comando CREATE TABLE permite definir distintos tipos de restricciones sobre una tabla: claves primarias, claves ajenas, obligatoriedad, valores por defecto y verificación de condiciones.

.De forma alternativa, podemos gestionar las constraints de una tabla mediante las sentencias de modificación de tablas.

Definición.

.Cláusula **CONSTRAINT**.

.Hay dos modos de especificar restricciones:

-**Restricción de columna.** En la definición de una columna al final de cada columna.

-**Restricción de tabla.** Definida como en una línea aparte.

UT 03. El Lenguaje DDL. Tablas y Constraints.

5.- Restricciones (Constraints).

5.1.- Restricciones definidas a nivel de columna. Definición.

•La restricción se define en la columna **después de definir el tipo de dato de la columna**.

•Se usa cuando:

•**La restricción está tan solo relacionada con la columna en cuestión.**

•Se desea desligar la definición del tipo de dato con las condiciones que deben cumplir.

•El formato de la definición es el siguiente:

```
CREATE TABLE NOMBRE_TABLA
(
    COLUMNA TIPODATO [CONSTRAINT NOMBRERESTRICCIÓN]
    [NOT NULL] [UNIQUE] [PRIMARY KEY] [DEFAULT VALOR]
    [REFERENCES NOMBRETABLA [(COLUMNA [, COLUMNA)]]]
    [ON DELETE CASCADE]]
    [CHECK (CONDICIÓN)],

    COLUMNA TIPODATO [CONSTRAINT NOMBRERESTRICCIÓN]
    [NOT NULL] [UNIQUE] [PRIMARY KEY] [DEFAULT VALOR]
    [REFERENCES NOMBRETABLA [(COLUMNA [, COLUMNA)]]]
    [ON DELETE CASCADE]]
    [CHECK (CONDICIÓN)],

    ...

    COLUMNA TIPODATO [CONSTRAINT NOMBRERESTRICCIÓN][RESTRICCIONES]
);
```

UT 03. El Lenguaje DDL. Tablas y Constraints.

5.- Restricciones (Constraints).

5.1.- Restricciones definidas a nivel de columna. Nombres de constraints.

.En la medida de lo posible, hay que poner nombres a las constraints de una tabla para poder identificar un error ya que, si se produce una violación de restricción, el SGBD informará de que restricción se ha violado.

.Se puede poner cualquier nombre a la restricción, usándose varias nomenclaturas. Una puede ser la siguiente: **CONSTRAINT tabla_campo_tipo**.

```
CREATE TABLE USUARIOS (  
    USERID NUMBER(6) CONSTRAINT USUARIOS_USERID_PK PRIMARY KEY,  
    NOMBRE VARCHAR2(20)  
);
```

Table USUARIOS creado.

UT 03. El Lenguaje DDL. Tablas y Constraints.

5.- Restricciones (Constraints).

5.1.- Restricciones definidas a nivel de columna. Constraint Not NULL (NN).

Se asocia a una sola columna y **se obliga a que ésta tenga un valor**. Es decir, prohíbe los valores nulos en esa columna. **Se puede definir la constraint con nombre o sin nombre.**

**CAMPO TIPODATO NOT NULL,
CAMPO TIPODATO CONSTRAINT NOMBRE NOT NULL,**

```
CREATE TABLE USUARIOS (  
    LOGIN VARCHAR2(15),  
    PASSWORD VARCHAR2(8) CONSTRAINT USUARIOS_PASSWORD_NN NOT NULL,  
);
```

Table USUARIOS creado.

DESC USUARIOS;

Nombre	¿Nulo?	Tipo
--------	--------	------

LOGIN	NOT NULL	VARCHAR2(15)
-------	----------	--------------

PASSWORD	NOT NULL	VARCHAR2(8)
----------	----------	-------------

```
INSERT INTO USUARIOS VALUES ('plopez', NULL);
```

Informe de error -

ORA-01400: cannot insert NULL into ("PRUEBA"."USUARIOS"."PASSWORD")

UT 03. El Lenguaje DDL. Tablas y Constraints.

5.- Restricciones (Constraints).

5.1.- Restricciones definidas a nivel de columna. Constraint Unique (UK).

Se asocia a una sola columna y **se obliga a que ésta tenga un valor**. Es decir, prohíbe los valores nulos en esa columna. **Se puede definir la constraint con nombre o sin nombre**.

**CAMPO TIPODATO UNIQUE,
CAMPO TIPODATO CONSTRAINT NOMBRE UNIQUE,**

```
CREATE TABLE COMPETIDORES (  
    NOMBRE VARCHAR2(30),  
    POSICION NUMBER(4) CONSTRAINT COMPETIDORES_POSICION_UK UNIQUE  
);
```

Table COMPETIDORES creado.

DESC COMPETIDORES;

Nombre	¿Nulo?	Tipo
-----	-----	-----
NOMBRE		VARCHAR2(30)
POSICION		NUMBER(4)

```
INSERT INTO COMPETIDORES VALUES ('JOSE PEREZ', 15);
```

1 fila insertadas.

```
INSERT INTO COMPETIDORES VALUES ('ANA GONZALEZ', 15);
```

Informe de error -

ORA-00001: unique constraint (**PRUEBA.COMPETIDORES_POSICION_UK**) violated

UT 03. El Lenguaje DDL. Tablas y Constraints.

5.- Restricciones (Constraints).

5.1.- Restricciones definidas a nivel de columna. Constraint Primary Key (PK).

• En el modelo relacional todas las tablas deben tener una clave primaria y sólo puede existir una clave primaria por cada tabla, aunque puede estar compuesta por varios atributos.

• Esta clave primaria podrá ser referenciada como clave ajena en otras tablas.

**CAMPO TIPODATO PRIMARY KEY,
CAMPO TIPODATO CONSTRAINT NOMBRE PRIMARY KEY,**

```
CREATE TABLE COMPETIDORES (  
    COMPETIDORID NUMBER(4) CONSTRAINT COMPETIDORES_COMPETIDORID_PK PRIMARY KEY,  
    NOMBRE VARCHAR2(30),  
    POSICION NUMBER(4) CONSTRAINT COMPETIDORES_POSICION_UK UNIQUE  
);
```

Table COMPETIDORES creado.

DESC COMPETIDORES;

Nombre	¿Nulo?	Tipo
-----	-----	-----
COMPETIDORID	NOT NULL	NUMBER(4)
NOMBRE		VARCHAR2(30)
POSICION		NUMBER(4)

```
INSERT INTO COMPETIDORES VALUES (1,'JOSE PEREZ', 15);
```

1 fila insertadas

```
INSERT INTO COMPETIDORES VALUES (1, 'ANA GONZALEZ', 17);
```

Informe de error -

ORA-00001: unique constraint (**PRUEBA.COMPETIDORES_COMPETIDORID_PK**) violated

UT 03. El Lenguaje DDL. Tablas y Constraints.

5.- Restricciones (Constraints).

5.1.- Restricciones definidas a nivel de columna. Constraint Foreign Key (FK) (I).

Una clave ajena, secundaria o foránea es un campo de una tabla que se relaciona con la clave primaria (o con la candidata de otra tabla) de la base de datos. Esta referencia se hace indicando los campos de donde procede.

Nota Importante. La constraint se expresa en varias líneas para que sea más visual y más legible. Puede estar escrita en una sola línea.

```
CAMPO TIPODATO REFERENCES TABLA_EXTERNA(CAMPO REFERENCIADO),
```

```
CAMPO TIPODATO CONSTRAINT NOMBRE  
REFERENCES TABLA_EXTERNA(CAMPO REFERENCIADO),
```

Al establecer la relación entre dos tablas, **la tabla que es referenciada debe estar creada**. Por tanto, **se crean primero se crearán las tablas que no tengan claves ajenas**.

Si queremos borrar las tablas tendremos que proceder al contrario, **borraremos primero las tablas que tengan las claves ajenas**. Este proceso puede solventarse añadiendo a la constraint FOREIGN KEY una cláusula **ON DELETE**. Existen dos posibilidades:

ON DELETE CASCADE. Permite borrar todos los registros cuya clave ajena sea igual a la clave del registro borrado.

ON DELETE SET NULL. Modifica el valor a NULL en todas las claves ajenas relacionadas con la borrada.

UT 03. El Lenguaje DDL. Tablas y Constraints.

5.- Restricciones (Constraints).

5.1.- Restricciones definidas a nivel de columna. Constraint Foreign Key (FK) (II).

```
CREATE TABLE PELICULAS (  
    PELICULAID NUMBER(6) CONSTRAINT PELICULAS_PELICULAID_PK PRIMARY KEY,  
    NOMBRE VARCHAR2(40),  
    DIRECTOR VARCHAR2(30)  
);
```

Table PELICULAS creado.

```
CREATE TABLE SOCIOS (  
    SOCIOID NUMBER(6) CONSTRAINT SOCIOS_SOCIOID_PK PRIMARY KEY,  
    NOMBRE VARCHAR2(20),  
    FECHANACIMIENTO DATE  
);
```

Table SOCIOS creado.

```
CREATE TABLE ALQUILERES (  
    ALQUILERID NUMBER(6) CONSTRAINT ALQUILERES_ALQUILERID_PK PRIMARY KEY,  
    PELICULAID NUMBER(6) CONSTRAINT ALQUILERES_PELICULAID_FK  
                                REFERENCES PELICULAS(PELICULAID),  
    SOCIOID NUMBER(6) CONSTRAINT ALQUILERES_SOCIOID_FK  
                                REFERENCES SOCIOS(SOCIOSID),  
    FECHAALQUILER DATE  
);
```

Table ALQUILERES creado.

UT 03. El Lenguaje DDL. Tablas y Constraints.

5.- Restricciones (Constraints).

5.1.- Restricciones definidas a nivel de columna. Constraint CHECK (CK).

La restricción **CHECK** permite comprobar que los valores que se introducen en un campo son adecuados evaluando una condición asociada a dicha columna. .

CAMPO TIPODATO CHECK(CONDICION),
CAMPO TIPODATO CONSTRAINT NOMBRE CHECK(CONDICION),

```
CREATE TABLE CLIENTES (  
    CLIENTEID NUMBER(5) CONSTRAINT CLIENTES_CLIENTEID_PK PRIMARY KEY,  
    NOMBRE VARCHAR2(30),  
    CREDITO NUMBER(4) CONSTRAINT CLIENTES_CREDITO_CK  
        CHECK (CREDITO BETWEEN 0 AND 2000)  
);
```

Table CLIENTES creado.

```
INSERT INTO CLIENTES VALUES (1, 'CENIPSA', 5000);
```

Informe de error -

ORA-02290: check constraint (PRUEBA.CLIENTES_CREDITO_CK) violated

UT 03. El Lenguaje DDL. Tablas y Constraints.

5.- Restricciones (Constraints).

5.2.- Restricciones definidas a nivel de tabla. Definición y Utilidad.

.Hay casos en los cuales no es posible o no se desea definir la constraint a nivel de columna. Se pueden dar las siguientes condiciones:

.Se desea poder tener la posibilidad de eliminar o bloquear la constraint sin tener redefinir el campo que la afecta.

.Es necesario definir una restricción que involucre a varios campos a la vez.

.En este caso, se definen las constraints como **constraints de tabla**.

.Una constraint de tabla se define en una línea por separado después de haber definido todos los campos de la tabla.

.Algunos diseñadores de bases de datos también prefieren definir así las constraints para tener más hacer más visual el código y depurarlo de forma más rápida.



UT 03. El Lenguaje DDL. Tablas y Constraints.

5.- Restricciones (Constraints).

5.2.- Restricciones definidas a nivel de tabla. Ejemplo.

```
CREATE TABLE PELICULAS (  
    PELICULAID NUMBER(6),  
    NOMBRE VARCHAR2(40),  
    DIRECTOR VARCHAR2(30),  
    CONSTRAINT PELICULAS_PELICULAID_PK PRIMARY KEY (PELICULAID)  
);
```

Table CLIENTES creado.

```
CREATE TABLE SOCIOS (  
    SOCIOID NUMBER(6),  
    NOMBRE VARCHAR2(20),  
    FECHANACIMIENTO DATE,  
    CONSTRAINT SOCIOS_SOCIOID_PK PRIMARY KEY (SOCIOID)  
);
```

Table CLIENTES creado.

```
CREATE TABLE ALQUILERES (  
    ALQUILERID NUMBER(6),  
    PELICULAID NUMBER(6),  
    SOCIOID NUMBER(6),  
    FECHAALQUILER DATE,  
    CONSTRAINT ALQUILERES_ALQUILERID_PK PRIMARY KEY (ALQUILERID),  
    CONSTRAINT ALQUILERES_PELICULAID_FK FOREIGN KEY (PELICULAID)  
        REFERENCES PELICULAS(PELICULAID),  
    CONSTRAINT ALQUILERES_SOCIOID_FK FOREIGN KEY (SOCIOID)  
        REFERENCES SOCIOS(SOCIOID)  
);
```

Table CLIENTES creado.

UT 03. El Lenguaje DDL. Tablas y Constraints.

5.- Restricciones (Constraints).

5.3.- Listado de Constraints de una Tabla.

• Para obtener un listado de constraints que están actuando en una tabla, se dispone se puede realizar una consulta a la tabla “**USER_CONSTRAINTS**” del diccionario de datos.

• Una posible consulta sería la siguiente:

```
SELECT CONSTRAINT_NAME, CONSTRAINT_TYPE, TABLE_NAME  
FROM USER_CONSTRAINTS;
```

• El campo **CONSTRAINT_TYPE** indica el tipo de constraint que es. Puede tomar los siguientes valores:

• **P.** PRIMARY KEY.

• **N.** NOT NULL.

• **R.** FOREIGN KEY.

• **U.** UNIQUE.

• **C.** CHECK.

UT 03. El Lenguaje DDL. Tablas y Constraints.

6.- Eliminación y vaciado de tablas.

6.1.- Eliminado (Borrado) de una Tabla en el Sistema.

.Para eliminar una tabla utilizaremos el comando **DROP TABLE** con la sintaxis:

```
DROP TABLE nombreTabla [CASCADE CONSTRAINTS];
```

.Con esta instrucción, **se borra la tabla con sus datos y cualquier información que existiera de ella en el diccionario de datos.**

.Opción CASCADE CONSTRAINTS. Las restricciones definidas donde un campo de la tabla que se desea borrar es clave ajena se borrarán antes y a continuación se eliminará la tabla.

.La operación de borrado en cascada es una opción muy peligrosa de realizar ya que hay que conocer el alcance de la operación para poder hacerla de una forma segura.

.Si no se especifica la opción CASCADE CONSTRAINTS y se pone en peligro la integridad referencial aparecerá un mensaje del tipo:

“ORA-02449: unique/primary keys in table referenced by foreign keys”.

```
CREATE TABLE USUARIO (  
    USUARIOID NUMBER(4) PRIMARY KEY,  
    NOMBRE VARCHAR2(30)  
);
```

Table USUARIO creado.

```
DROP TABLE USUARIO;
```

Table USUARIO borrado.

UT 03. El Lenguaje DDL. Tablas y Constraints.

6.- Eliminación y vaciado de tablas.

6.2.- Vaciado de Datos y Renombrado de una Tabla en el Sistema.

Vaciado de una Tabla.

.El comando **TRUNCATE TABLE** permite borrar las filas de una tabla sin eliminar su estructura. La sintaxis es la siguiente:

```
TRUNCATE TABLE nombreTabla;
```

Renombrar una tabla.

.Si se desea cambiar el nombre de la tabla sin eliminar los datos se dispone del comando **RENAME TABLE** que permite borrar las filas de una tabla sin eliminar su estructura. La sintaxis es la siguiente:

```
RENAME nombreTabla TO nuevoNombre;
```

UT 03. El Lenguaje DDL. Tablas y Constraints.

7.- Modificación de la Estructura de una Tabla.

7.1.- Modificación de los Campos de una Tabla. Añadir columnas a una tabla.

•Podemos añadir uno o varios campos añadiendo la cláusula ADD y a continuación la definición de los nuevos campos. El formato es el siguiente:

```
ALTER TABLE NOMBRETABLA ADD (  
    COLUMNANUEVA1 TIPODATOS1 [PROPIEDADES],  
    COLUMNANUEVA2 TIPODATOS2 [PROPIEDADES],  
    ...  
);
```

```
CREATE TABLE USUARIOS (  
    CREDITO NUMBER(4) CHECK (CREDITO BETWEEN 0 AND 2000)  
);
```

Table USUARIOS creado.

```
ALTER TABLE USUARIOS ADD (  
    USUARIOID NUMBER(6) PRIMARY KEY,  
    NOMBRE VARCHAR2(20)  
);
```

DESC usuarios;

Nombre	¿Nulo?	Tipo
-----	-----	-----
credito		NUMBER(4)
usuarioID	NOT NULL	NUMBER(6)
nombre		VARCHAR2(20)

UT 03. El Lenguaje DDL. Tablas y Constraints.

7.- Modificación de la Estructura de una Tabla.

7.1.- Modificación de los Campos de una Tabla. Cambiar nombre de una columna.

Cambiar el nombre a una columna.

•Podemos añadir uno o varios campos añadiendo la clausula ADD y a continuación la definición de los nuevos campos.

•El formato es el siguiente:

```
ALTER TABLE NOMBRETABLA  
RENAME COLUMN NOMBREVIEJO TO NOMBRENUEVO;
```

```
ALTER TABLE USUARIOS  
RENAME COLUMN NOMBRE TO LOGIN;
```

```
DESC USUARIOS;
```

Nombre	¿Nulo?	Tipo
-----	-----	-----
credito		NUMBER(4)
usuarioID	NOT NULL	NUMBER(6)
login		VARCHAR2(20)

UT 03. El Lenguaje DDL. Tablas y Constraints.

7.- Modificación de la Estructura de una Tabla.

7.1.- Modificación de los Campos de una Tabla. Eliminar columnas a una tabla.

Eliminar columnas de una tabla.

.Se eliminará la columna indicada **sin poder deshacer esta acción**. Además, **se eliminarán todos los datos que contuviera**.

.Es posible eliminar **varios campos simultáneamente**, no pudiéndose dejar una tabla sin **campos**; en este caso, sería necesario borrar la tabla.

.El formato es el siguiente:

```
ALTER TABLE NOMBRETABLA DROP (  
    COLUMNA1,  
    COLUMNA2,  
    ....  
);
```

UT 03. El Lenguaje DDL. Tablas y Constraints.

7.- Modificación de la Estructura de una Tabla.

7.1.- Modificación de los Campos de una Tabla. Modificar columnas.

Modificar columnas de una tabla.

.Podemos modificar el tipo de datos o las propiedades de la columna.

.Todos estos cambios **son posibles si la tabla no contiene datos o si los datos guardados no ocupan todo el espacio reservado para ellos**. Es decir, si la columna contiene datos es imposible truncar números o cadenas de texto.

.El formato es el siguiente:

```
ALTER TABLE NOMBRETABLA MODIFY (  
    COLUMNA1 TIPODATO1 [PROPIEDADES],  
    COLUMNA2 TIPODATO2 [PROPIEDADES],  
    ...  
);
```

```
CREATE TABLE EMPLEADOS (  
    EMPLEAID NUMBER(6) PRIMARY KEY,  
    NOMBRE VARCHAR2(20)  
);  
  
ALTER TABLE EMPLEADOS MODIFY (  
    NOMBRE VARCHAR2(30)  
);
```

En este caso no habría ningún problema en realizar la operación ya que se amplía el campo texto. Si fuera necesario disminuir el tamaño dependería de la longitud de los datos que contuviera el campo nombre.

UT 03. El Lenguaje DDL. Tablas y Constraints.

7.- Modificación de la Estructura de una Tabla.

7.2.- Modificación de Constraints. Añadir y Borrar Constraints.

Añadir restricciones a una tabla.

- Si la tabla está vacía, no hay ningún problema.
- Sin embargo, hay datos insertados en la tabla, es posible que los datos violen de por sí la restricción.
- El comando para añadir una CONSTRAINT es el siguiente:

```
ALTER TABLE NOMBRETABLA  
ADD CONSTRAINT NOMBRERESTRICCIÓN RESTRICCIÓN;
```

```
ALTER TABLE PEDIDOS  
ADD CONSTRAINT PEDIDOS_CLIENTES_FK FOREIGN KEY (CLIENTEID)  
REFERENCES T_CLIENTES(CLIENTEID);
```

Borrar restricción de una tabla.

- Para borrar una CONSTRAINT **se debe haber indicado un nombre a la restricción.**
- El comando es el siguiente:

```
ALTER TABLE NOMBRETABLA  
DROP CONSTRAINT NOMBRERESTRICCIÓN;
```

```
ALTER TABLE PEDIDOS  
DROP CONSTRAINT PEDIDOS_CLIENTES_FK;
```

UT 03. El Lenguaje DDL. Tablas y Constraints.

7.- Modificación de la Estructura de una Tabla.

7.2.- Modificación de Constraints. Modificar y Activar Restricciones.

Modificar restricciones de una tabla.

•Es imposible modificar una restricción ya creada. Se debe eliminar la existente y crearla con las condiciones que se desean incluir.

Modificar el nombre de una restricción.

```
ALTER TABLE NOMBRETABLA  
RENAME CONSTRAINT NOMBRECONSTRAINT TO NOMBRENUEVOCONSTRAINT;
```

Activar o desactivar restricciones.

•Es necesario desactivar las restricciones ya que se deben importar datos en una migración de una aplicación y es más cómodo que las restricciones no actúen.

•Es importante que los datos no presenten problemas de consistencia y de integridad referencial.

•Los comandos para desactivar y activar las constraints son:

```
ALTER TABLE TABLA DISABLE CONSTRAINT NOMBRERESTRICCIÓN [CASCADE];  
ALTER TABLE TABLA ENABLE CONSTRAINT NOMBRERESTRICCIÓN [CASCADE];
```

UT 03. El Lenguaje DDL. Tablas y Constraints.

8.- Índices. Creación y Eliminación de Índices.

- Crear índices ayuda a la localización más rápida de la información contenida en las tablas.
- Cuando se define una clave primaria, una clave secundaria o una restricción única automáticamente se crea un índice.
- Los índices son útiles en tablas de gran tamaño ya que aceleran las búsquedas de información.

Creación de un índice sobre campos de una tabla.

```
CREATE INDEX nombreIndice ON nombreTable (columna1, [, columna2 ...]);
```

Eliminación de un índice.

```
DROP INDEX nombreIndice;
```